

조운호

반복되는 인프라 운영과 배포 문제를 자동화하고, 책임 경계와 실패 복구 구조를 설계하는 신입 플랫폼 엔지니어

PLATFORM ENGINEER

Email	code.penguin.yoon@gmail.com
-------	--

GitHub	github.com/CodingPenguin-yoon
--------	---

Web	yoonman.page
-----	--

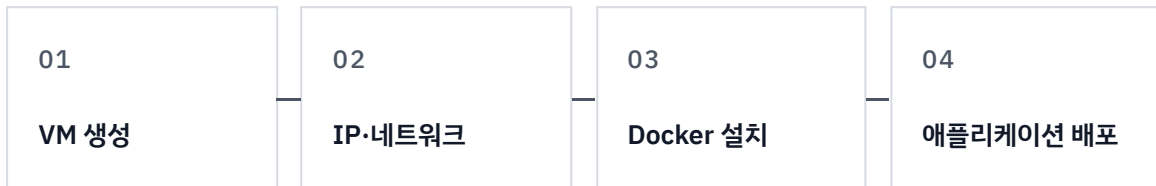
플랫폼 운영이라는 하나의 문제를 다뤘습니다.

배포 자동화, VM 생성, 실행 상태 추적을 서로 연결된 운영 문제로 풀었습니다.

배포 자동화 Heimdall 후보 실행 환경을 만들고 검증한 뒤 현재 세대로 승격합니다.	VM 생성 자동화 Gjallar 실제 Proxmox 상태를 기준으로 승인된 VM 생성 요청을 실행합니 다.	상태와 실패 추적 K-Le-PaaS 입력 해석부터 실행 결과까지 확인 가능한 흐름으로 연결합니다.
보조 프로젝트 Argus	공급자별 입력 경계를 분리하고 비교 가능한 형태로 정규 화한 보조 프로젝트입니다.	

반복 설정이 자동화의 출발점이었습니다.

VM 생성부터 IP·네트워크·Docker·배포까지 매번 반복되는 작업과 설정 편차를 줄이고 싶었습니다.



새 프로그램을 올릴 때마다 같은 순서로 환경을 준비했습니다.

수동 입력

새 환경마다 같은 값을 다시 입력해야 했습니다.

설정 편차

작업 순서와 설정이 실행할 때마다 달라질 수 있었습니다.

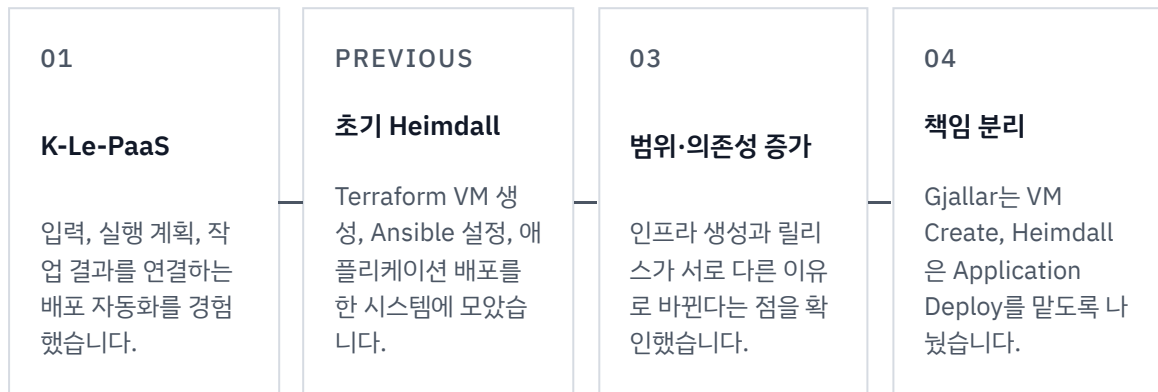
복구 어려움

어느 단계에서 실패했는지 확인하고 되돌리기 어려웠습니다.

홈랩을 반복되는 운영 문제를 확인하고 자동화를 검증하는 환경으로 사용했습니다.

큰 자동화 하나를 두 개의 책임으로 나눴습니다.

VM 생성과 애플리케이션 배포는 변경 주기와 실패 영향이 달랐습니다.



현재 책임 분리

Gjallar	Heimdall
VM Create	Application Deploy

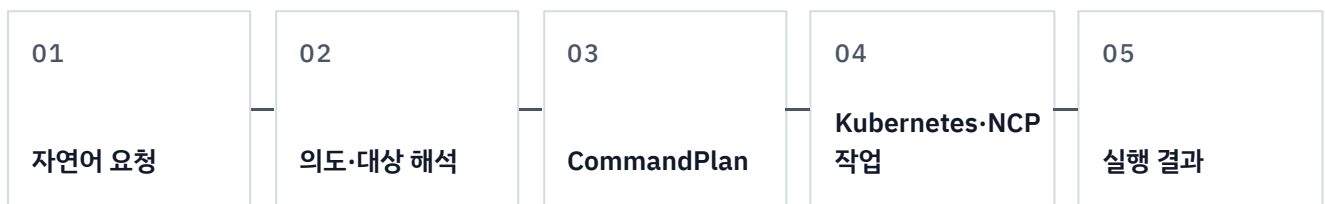
VM 생성과 애플리케이션 배포는 변경 주기가 달랐습니다.

한쪽 실패가 다른 쪽의 상태와 복구 경로에 미치는 영향도 달랐습니다.

2인 팀 프로젝트이며 팀 기능과 개인 기여를 분리해 설명합니다.

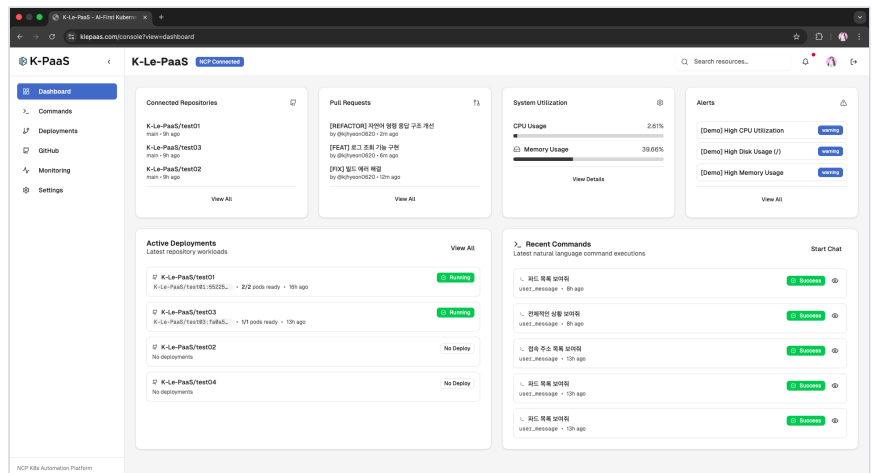
자동화는 실행보다 입력과 피드백까지 포함해야 했습니다.

자연어 요청을 Kubernetes 명령으로 바꾸고 결과를 되돌려주는 흐름에서 플랫폼 자동화의 기본 구조를 경험했습니다.



2인 팀 성과

- 자연어 요청을 의도와 대상으로 해석해 실행 가능한 CommandPlan으로 변환했습니다.
- 명령 계획을 Kubernetes·NCP 작업에 연결하고 상태와 결과를 기록했습니다.
- 영향이 큰 작업은 사용자 확인 뒤 실행하도록 대기 상태를 뒀습니다.



저장소 검증

실제 서비스 화면에서 배포 상태와 자연어 명령 실행 결과를 함께 확인했습니다.

public/projects/klepaas-dashboard.png

검증된 개인 기여

자연어 요청에서 실행 대상과 의도를 추출하는 Gemini 흐름을 구현했습니다.

Ingress 도메인 변경과 배포 URL 동기화 흐름을 맡았습니다.

여기서 얻은 학습

Pod·Service·Deployment 상태 조회와 재시작 명령을 CommandPlan으로 연결했습니다.

Prometheus 메트릭을 NKS 리소스 상태 화면에 연결했습니다.

입력 정규화

고위험 작업 확인

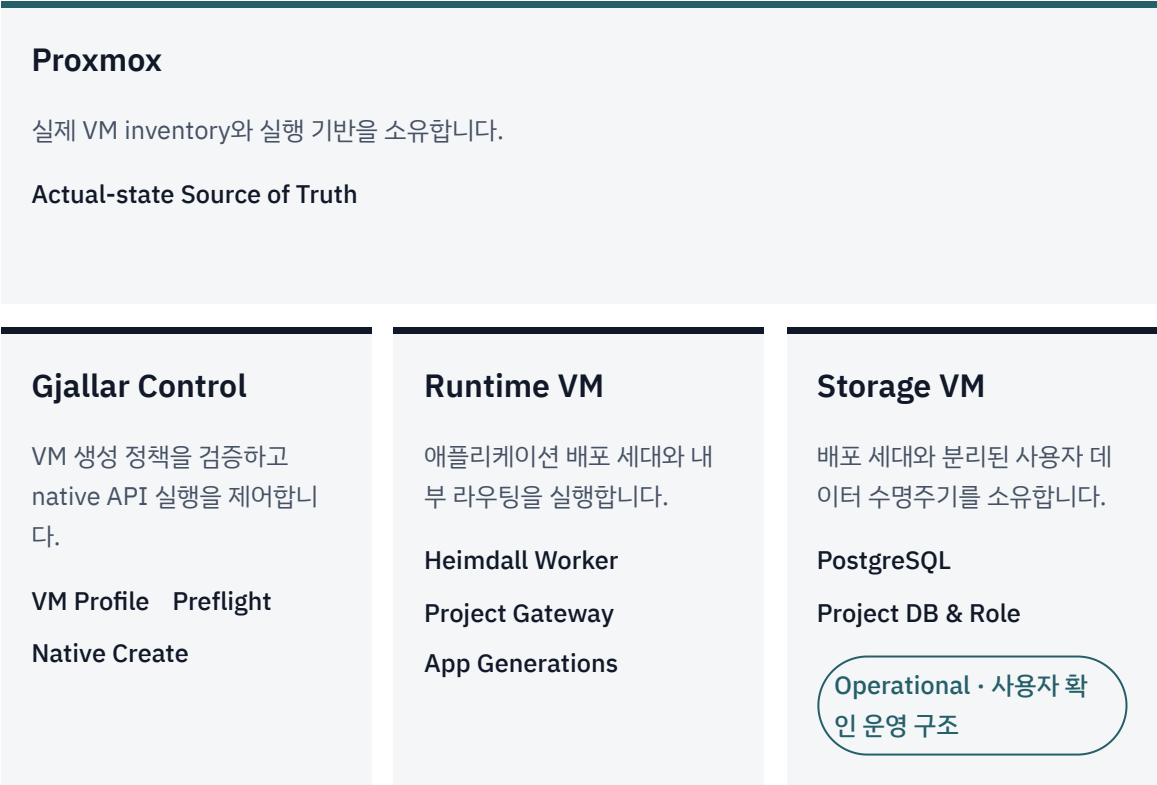
실행 피드백

Storage VM 운영은 사용자 확인 사항이며 저장소 근거와 분리해 표시합니다.

실행 기반, 배포, 데이터의 수명주기를 분리했습니다.

Proxmox 위에서 Gjallar는 VM을, Heimdall은 배포 세대를, Storage VM은 사용자 데이터를 책임집니다.

현재 시스템의 책임과 운영 연결



실선은 현재 코드로 확인한 경로입니다. Storage VM 연결은 별도 PostgreSQL VM을 사용한다는 사용자 확인 운영 구조입니다.

Lifecycle boundary 애플리케이션 실행 환경과 사용자 데이터의 수명주기를 분리했습니다.

같은 홈랩 안에서도 실패 경계는 달라야 했습니다.

인프라 생성 실패와 후보 배포 실패가 서로의 상태를 오염시키지 않도록 소유권을 나눴습니다.

Gjallar와 Heimdall의 현재 책임 경계

구분	Gjallar	Heimdall
책임	VM 생성 요청의 정책·검증·실행	애플리케이션 세대의 빌드·검증·승격
변경 이유	인프라 용량·구성 변경	애플리케이션 릴리스
실행 대상	Proxmox VM	Docker Deployment Generation
실패 영향	실행 기반 생성 실패	후보 배포 미승격
현재 범위	Native Create · 제한된 gated Start	Deployment Generation 생성·검증·Current 승격

Before / After

BEFORE · 초기 HEIMDALL

Terraform VM 생성, Ansible 설정, 애플리케이션 배포가 한 자동화에 집중됐습니다.

AFTER · 책임 분리

Gjallar의 VM Create와 Heimdall의 Deployment Generation으로 소유권을 나눴습니다.

현재 범위는 native Create VM과 제한된 gated operations입니다.

Proxmox의 실제 상태를 기준으로 VM 생성을 반복 가능하게 만들었습니다.

정책을 담은 VM Profile과 실제 Template Inventory를 분리하고, 승인된 요청만 native API로 실행합니다.

SOURCE OF TRUTH

Proxmox actual inventory

DB가 아닌 Proxmox 조회 결과를 현재 VM 상태의 기준으로 사용합니다.

POLICY

DB-backed VM Profile

CPU·메모리·디스크 범위와 access 권장값을 용도별 정책으로 재사용합니다.

PREFLIGHT

실행 전 조건 검증

고정 IP, bridge, storage, template과 guest-agent 준비 상태를 먼저 확인합니다.

GATED EXECUTION

명시적 승인 뒤 native API 실행

Draft → Preflight → Plan → Approval → Preview → Acknowledgement → Native Proxmox API

OUTCOME

기본은 stopped create

선택 시에만 부팅하고 guest-agent IP와 cloud-init 완료 근거를 확인합니다.

Gjallar Operations Console
Proxmox VM 운영 관리

admin ADMIN
Logout

Overview
VM Instances
DRS Advisor
Operations
Settings

Cluster OK LIVE READ-ONLY

Proxmox 클러스터 운영 화면
새로고침
VM Instances
Create VM

NODES
3/3
gjallar-mvp

VMS
18
11 running

STORAGE
NFS
5,212 GB free

RISKS
0
0 active jobs

DATACENTER
gjallar-mvp

- yoonsmserver
- yoonsmserver2
- yoonsmserver3
- Network readiness 6
- Jobs 0

Cluster Summary
노드별 VM 상태, 실시간 CPU/메모리 사용량, 스토리지 여유 공간, 브리지 상태
3/3 online

NODE	STATUS	VMS	CPU USAGE	MEMORY USAGE	NETWORK	STORAGE
yoonsmserver yoonsmserver	online	8/11 running / total	2%	48 / 60 GB	vmb1 / vmb0	1,828 GB free
yoonsmserver2 yoonsmserver2	online	3/7 running / total	0%	13 / 60 GB	vmb0 / vmb1	1,613 GB free
yoonsmserver3 yoonsmserver3	online	0/0 running / total	0%	2 / 60 GB	vmb0 / vmb1	1,771 GB free

저장소 검증

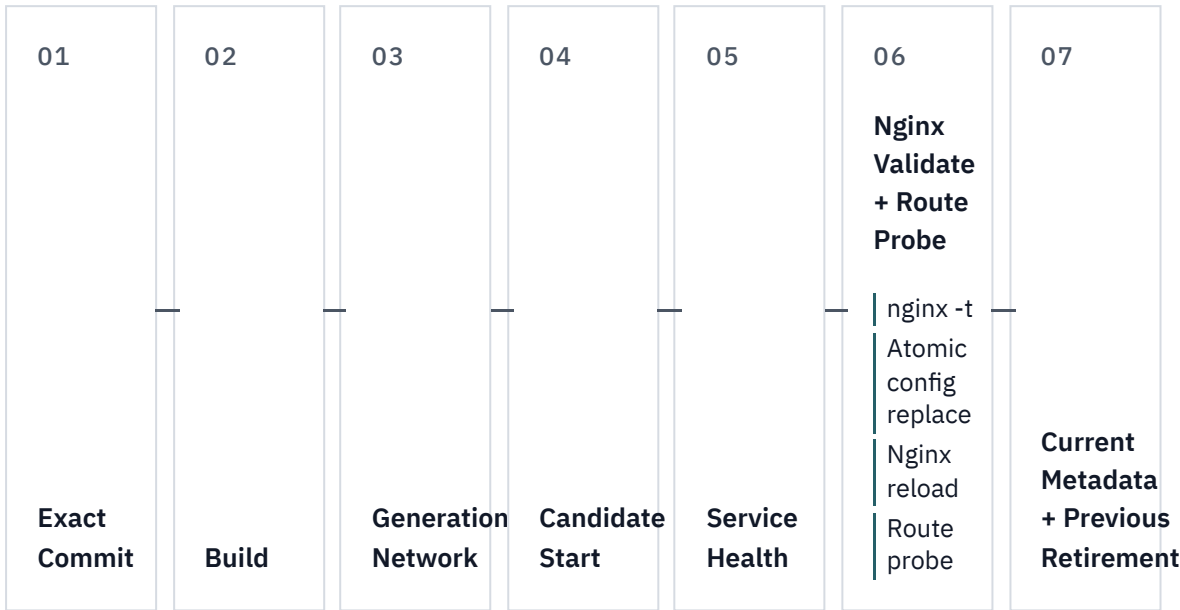
실제 운영 콘솔 화면은 Proxmox node·VM·storage inventory와 job 상태 조회 범위를 보여줍니다.
public/projects/gjallar.png

SCOPE LIMIT

현재 구현은 Native Create와 stopped VM의 제한된 gated Start입니다. 전체 VM lifecycle과 destructive control은 범위가 아닙니다.

검증된 세대만 Current로 승격합니다.

고정된 commit으로 후보 세대를 만든 뒤 health check와 실제 route probe를 모두 통과해야 트래픽을 전환합니다.



Candidate 실행 성공과 운영 트래픽 활성화 성공을 서로 다른 판정으로 기록합니다.

두 개의 성공 판정

EXECUTION SUCCESS

candidate가 시작되고 service health를 통과한 상태입니다.
아직 운영 트래픽 승격은 아닙니다.

Generation Isolation

세대별 network와 고유 service alias
로 Current와 Candidate의 실행 경계
를 분리합니다.

IMPLEMENTED

Project DB · Role

프로젝트별 PostgreSQL DB와 login
role을 만들고 연결을 검증합니다.

IMPLEMENTED

TRAFFIC ACTIVATION SUCCESS

Nginx 검증과 실제 route probe가 끝나 current metadata
를 전환할 수 있는 상태입니다.

Previous Retirement

Nginx 활성화와 실제 route probe 성
공이 확정된 후에만 이전 generation을
회수합니다.

IMPLEMENTED

Deploy-time Injection

DB 접근을 선언한 서비스에만 연결 정
보와 read-only secret을 전달합니다.

IMPLEMENTED

이미지·데이터 rollback과 DB purge·backup·restore는 구현 범위가 아닙니다.

실패할수록 보존 범위를 좁혔습니다.

활성화 실패에는 직전 정상 경로를 복원하고, 상태가 불확실할 때는 자동 삭제보다 보존을 선택합니다.

실패 유형별 정리와 보존 경계

Failure	Handling	Preserved	Limit
Build / Health	실패한 배포 ID와 label이 정확히 일치하는 candidate만 정리	기존 Current와 active metadata	저장된 image 재사용 없음 Stored-image release rollback
Nginx Activation	last-known-good 설정 복원 후 실패 candidate를 exact-label 정리	이전 route와 generation	release rollback 아님
Worker Interruption	Control DB·Nginx marker·Docker label을 비교해 reconcile	상태가 불확실한 candidate	확정할 수 없으면 수동 판단
App Deployment / Data	runtime generation과 PostgreSQL 데이터 실패 범위를 분리	별도 Storage VM의 사용자 데이터	DB purge·backup·restore와 data rollback 비범위 Database backup · restore Database purge · rotation Database credential rotation User data rollback

Unknown Runtime State

Worker가 중단된 뒤 runtime 상태를 확정할 수 없으면 자동 삭제보다 candidate 보존을 선택합니다.

PRESERVE

PostgreSQL Boundary

프로젝트별 DB·role provisioning과 배포 연동은 구현됐습니다. 별도 PostgreSQL Storage VM은 사용자 확인 운영 구조이며 기본 Compose의 물리 격리를 뜻하지 않습니다.

OPERATIONAL

외부 Edge는 고정하고 동적 라우팅은 내부에서 처리합니다.

Control Web과 배포 도메인을 분리하고 OCI Edge에서 홈랩 내부 Ingress까지 하나의 중계 경계를 둡니다.

고정 외부 EDGE와 내부 동적 ROUTING 계획



모든 점선은 계획 연결입니다. 실제 도메인·IP·방화벽 규칙·secret·certificate는 이 설계에 포함하지 않습니다.

Dynamic Route Owner OCI Edge는 배포마다 바뀌지 않습니다. deployment별 동적 routing은 Internal Ingress와 Project Gateway가 소유합니다.

다른 문제에도 입력 경계를 분리하는 원칙을 적용했습니다.

서로 다른 데이터 공급자를 독립적으로 수집하고 정규화해 비교 가능한 판단 흐름으로 만들었습니다.

Provider Adapter → Normalized Snapshot → Judgement → Dashboard



조운호 · Platform Engineer

운영 문제를 발견하고 자동화한 뒤, 책임과 실패 경계를 다시 설계할 수 있습니다.

Selected Project Scope

01 Heimdall

Implemented · Generation Deployment

고정 commit의 candidate를 격리하고 health와 실제 route를 검증한 뒤에만 current로 승격합니다.

02 Gjallar

Implemented · Native Create VM

Proxmox inventory를 기준으로 preflight와 명시적 승인을 거쳐 native API로 VM을 생성합니다.

03 K-Le-PaaS

2인 팀 · Verified Personal Scope

팀 안에서 맡은 자연어 해석과 CommandPlan 구현으로 Kubernetes 작업과 실행 피드백을 연결했습니다.

04 Argus

Implemented · Supporting Project

공급자 adapter와 normalized snapshot 경계를 분리해 수집 입력을 하나의 판단 흐름으로 연결했습니다.

Core Technologies

Platform / Infrastructure

Proxmox · Kubernetes · NCP · Docker · Linux

Backend / Data

FastAPI · PostgreSQL · Redis · SQLAlchemy · REST API

Interface / Delivery

React · Next.js · TypeScript · GitHub Actions · WireGuard

Education · Certification · Contact

EDUCATION

광운대학교

전자통신공학과 · 졸업 · 2026.02

CERTIFICATION

정보처리기사

리눅스마스터 2급

Email code.penguin.yoon@gmail.com

GitHub github.com/CodingPenguin-yoon

Web yoonman.page